

Tareas Técnicas — Yandri Alcivar

Resumen completo de todas las tareas técnicas ejecutadas por Yandri durante el Sprint 1 del proyecto Alpha Inmobiliaria.

TT-001: Configuración del Monorepo

Objetivo: Crear la estructura base del proyecto con monorepo.

Archivos creados:

- `package.json` raíz con workspaces `["apps/*"]`
- `apps/backend/package.json` con dependencias: Express 5, TypeScript 6, Prisma 7, Zod 4, etc.
- `apps/backend/tsconfig.json` — configuración de TypeScript
- `apps/backend/.env` — variables de entorno
- `apps/backend/src/server.ts` — servidor Express con endpoint `/api/health`
- `.gitignore`

Logros:

- Monorepo funcional con `npm run dev` desde `apps/backend`
- Endpoint health check operativo
- Repositorio inicializado y subido a GitHub (`chaoslov/proyecto_bienesyraices1`)
- Git workflow establecido: `feature/nombre-tt-00X` → PR → merge a `main`

Documentación: `docs/TT-001-configuracion-monorepo.md`

TT-002: Esquema Prisma y Modelo de Datos

Objetivo: Diseñar e implementar el modelo de datos completo.

Archivos creados:

- `apps/backend/prisma/schema.prisma` — 7 entidades, 5 enums
- Migración inicial (`prisma/migrations/`)

Entidades:

Entidad	Campos Clave
User	id, email (único), password, rol, activo
Asesor	id, userId, nombre, telefono, foto, especialidad, etc.
Propiedad	id, titulo, descripcion, precio, tipoPropiedad, tipoTransaccion, etc.
Ubicacion	id, direccion, sector, ciudad, latitud, longitud
Imagen	id, url, publicId, orden

Entidad	Campos Clave
Mensaje	id, nombre, email, mensaje, tipo, leído, archivado

Enums: Rol, TipoPropiedad, TipoTransaccion, EstadoPropiedad, TipoMensaje

Decisiones técnicas:

- User y Asesor separados (SOLID) — User maneja auth, Asesor datos profesionales
- Generador `prisma-client` (Prisma 7)
- `@prisma/adapters-pg` para conexión PostgreSQL

Documentación: docs/TT-002-esquema-prisma-modelo-datos.md

TT-003: Conexión PostgreSQL (Supabase)

Objetivo: Conectar el backend a PostgreSQL usando Supabase.

Archivos creados/modificados:

- `apps/backend/src/infrastructure/database/prisma.ts` — cliente Prisma con `@prisma/adapters-pg` + `pg`
- `apps/backend/.env` — credenciales de conexión

Logros:

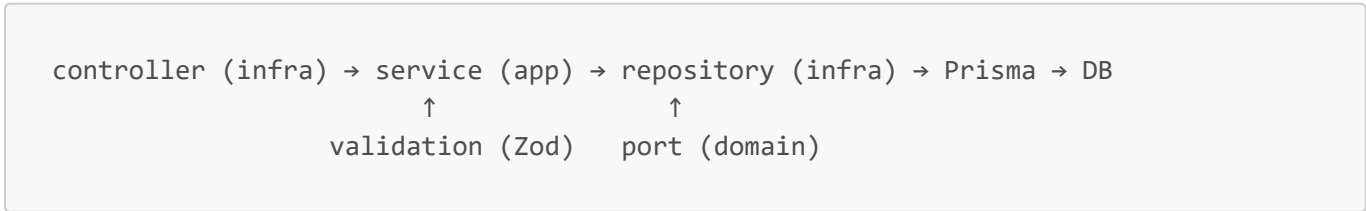
- Conexión exitosa mediante Transaction Pooler de Supabase
- IPv4 add-on habilitado para compatibilidad con pooler
- Parámetros de conexión pasados por separado (no URL string) para evitar problemas de parsing
- Health check con verificación de DB: `GET /api/health` → `{ "status": "ok", "db": "conectado" }`
- `prisma migrate dev --name init` creó todas las tablas en Supabase

Documentación: docs/TT-003-conexion-postgresql-express.md

TT-004: CRUD de Propiedades

Objetivo: Implementar el CRUD completo de Propiedades siguiendo arquitectura hexagonal.

Arquitectura hexagonal (11 archivos):



Endpoints creados (7):

Método	Ruta	Propósito
GET	/api/propiedades	Listar con filtros y paginación

Método	Ruta	Propósito
GET	/api/propiedades/destacadas	Propiedades destacadas
GET	/api/propiedades/:id	Detalle con ubicación e imágenes
POST	/api/propiedades	Crear con ubicación anidada
PUT	/api/propiedades/:id	Actualizar
PATCH	/api/propiedades/:id/estado	Cambiar estado
DELETE	/api/propiedades/:id	Eliminar

Archivos creados:

1. `domain/ports/IPropiedadRepository.ts` — puerto/interfaz
2. `application/validations/propiedad.validation.ts` — Zod schemas
3. `application/services/PropiedadService.ts` — lógica de negocio
4. `application/dtos/requests/CreatePropiedadRequest.ts`
5. `application/dtos/requests/UpdatePropiedadRequest.ts`
6. `application/dtos/requests/FiltrosPropiedadRequest.ts`
7. `application/dtos/responses/PropiedadResponse.ts`
8. `infrastructure/repositories/PropiedadRepository.ts` — implementación Prisma
9. `infrastructure/controllers/PropiedadController.ts` — handlers HTTP
10. `infrastructure/routes/propiedad.routes.ts` — rutas
11. `infrastructure/middlewares/validateId.ts` — helper `getId(req)`

Problema encontrado y solucionado: `req.params.id` tipado como `string | string[]` en Express — resuelto con helper `getId(req)` que envuelve en `String()`.

Documentación: `docs/TT-004-crud-propiedades.md`

TT-005: CRUD de Asesores

Objetivo: Implementar el CRUD completo de Asesores con arquitectura hexagonal.

Endpoints creados (5):

Método	Ruta	Propósito
GET	/api/asesores	Listar (incluye <code>_count</code> de propiedades y mensajes)
GET	/api/asesores/:id	Detalle por ID
POST	/api/asesores	Crear (User + Asesor en transacción)
PUT	/api/asesores/:id	Actualizar perfil
DELETE	/api/asesores/:id	Eliminar en cascada

Características importantes:

- `POST /api/asesores` usa `prisma.$transaction` para crear `User` + `Asesor` atómicamente
- Si falla un paso, se revierte todo (evita usuarios huérfanos)
- `DELETE` elimina en orden: mensajes → propiedades → asesor → user (por las FK)
- Contraseña cifrada con `bcryptjs` (10 salt rounds)

Archivos creados (9):

1. `domain/ports/IASesorRepository.ts`
2. `application/validations/asesor.validation.ts`
3. `application/services/AsesorService.ts`
4. `application/dtos/requests/CreateAsesorRequest.ts`
5. `application/dtos/requests/UpdateAsesorRequest.ts`
6. `application/dtos/responses/AsesorResponse.ts`
7. `infrastructure/repositories/AsesorRepository.ts`
8. `infrastructure/controllers/AsesorController.ts`
9. `infrastructure/routes/asesor.routes.ts`

Más `server.ts` — agregado `app.use('/api', asesorRouter)`

Documentación: `docs/TT-005-crud-asesores.md`

TT-005.5: Refactor de Arquitectura Hexagonal

Objetivo: Corregir violaciones estructurales detectadas en revisión de código.

Problemas corregidos:

#	Violación	Solución
a	Service importaba repositorio concreto (singleton)	Constructor recibe interfaz <code>I*Repository</code>
b	Puerto importaba <code>Prisma</code> del generado	Eliminado el import (no se usaba)
c	Repository no usaba <code>implements</code>	Agregado <code>implements I*Repository</code>
d	Todo era <code>any</code> , sin entidades de dominio	Creadas <code>PropiedadEntity</code> y <code>AsesorEntity</code>
e	Singletons impedían testear con mocks	Servicios reciben repositorio por constructor

Archivos creados (2):

- `domain/entities/Propiedad.ts` — `PropiedadEntity, PaginatedResult<T>`
- `domain/entities/Asesor.ts` — `AsesorEntity`

Archivos modificados (8):

- `IPropiedadRepository.ts` — eliminado import de `Prisma`
- `PropiedadRepository.ts` — `implements IPropiedadRepository`, eliminado singleton
- `AsesorRepository.ts` — `implements IAsesorRepository`, eliminado singleton
- `PropiedadService.ts` — DI por constructor
- `AsesorService.ts` — DI por constructor
- `PropiedadController.ts` — DI por constructor, métodos de instancia

- `AsesorController.ts` — DI por constructor, métodos de instancia
- `propiedad.routes.ts` / `asesor.routes.ts` — wiring de dependencias

Nuevo flujo:

```
routes (wiring)
  → new Repositorio()
  → new Servicio/repositorio)      ← solo conoce interfaz
  → new Controlador(servicio)      ← solo conoce clase del servicio
```

Documentación: `docs/TT-005.5-refactor-hexagonal.md`

TT-006: API de Mensajes y Subida de Imágenes a Cloudinary

Objetivo: Crear CRUD de mensajes y sistema de subida de imágenes a Cloudinary.

Parte 1: CRUD de Mensajes

Endpoints (5):

Método	Ruta	Propósito
POST	<code>/api/mensajes</code>	Enviar mensaje (público)
GET	<code>/api/mensajes/asesor/:asesorId</code>	Listar por asesor
PATCH	<code>/api/mensajes/:id/leer</code>	Marcar leído
PATCH	<code>/api/mensajes/:id/archivar</code>	Archivar
DELETE	<code>/api/mensajes/:id</code>	Eliminar

Parte 2: Subida de Imágenes a Cloudinary

Configuración: Cloudinary SDK con `memoryStorage` de Multer (5MB max, JPEG/PNG/WebP)

Endpoints (3):

Método	Ruta	Propósito
POST	<code>/api/propiedades/:id/imagenes</code>	Subir 1 imagen
POST	<code>/api/propiedades/:id/imagenes/multiples</code>	Subir múltiples (máx 10)
DELETE	<code>/api/imagenes/:id</code>	Eliminar de Cloudinary + DB

Dependencias nuevas: `cloudinary`, `multer`, `@types/multer`

Archivos creados (13):

1. `domain/ports/IMensajeRepository.ts`
2. `domain/ports/IImagenRepository.ts`

- 3. `application/validations/mensaje.validation.ts`
- 4. `application/services/MensajeService.ts`
- 5. `application/services/ImagenService.ts`
- 6. `infrastructure/repositories/MensajeRepository.ts`
- 7. `infrastructure/repositories/ImagenRepository.ts`
- 8. `infrastructure/controllers/MensajeController.ts`
- 9. `infrastructure/controllers/ImagenController.ts`
- 10. `infrastructure/routes/mensaje.routes.ts`
- 11. `infrastructure/routes/imagen.routes.ts`
- 12. `infrastructure/cloudinary/config.ts`
- 13. `infrastructure/middlewares/upload.ts`

Fix adicional: `PropiedadRepository.create` corregido para usar `connect` con `asesor` (Prisma 7 requiere relaciones explícitas).

Documentación: `docs/TT-006-api-mensajes-cloudinary.md`

TT-007: Autenticación JWT

Objetivo: Implementar login con JWT y middleware de protección de rutas.

Endpoints (2):

Método	Ruta	Auth	Propósito
POST	<code>/api/auth/login</code>	No	Login → token JWT (7 días) + usuario
GET	<code>/api/auth/me</code>	Sí	Obtener usuario actual desde token

Flujo de login:

- 1. Validar email + password con Zod
- 2. Buscar User + Asesor por email
- 3. Comparar password con `bcrypt.compare()`
- 4. Generar token JWT con `jwt.sign({ id, email, rol }, JWT_SECRET, expiresIn: '7d')`
- 5. Devolver { token, user }

Middleware creado:

- `requireAuth` — verifica `Authorization: Bearer <token>`, decodifica y adjunta `req.user`
- `requireRol(...roles)` — restringe rutas por rol (ej: `requireRol('admin')`)

Archivos creados (7):

- 1. `domain/ports/IUserRepository.ts`
- 2. `infrastructure/repositories/UserRepository.ts`
- 3. `application/validations/auth.validation.ts`
- 4. `application/services/AuthService.ts`
- 5. `infrastructure/controllers/AuthController.ts`

- 6. `infrastructure/routes/auth.routes.ts`
- 7. `infrastructure/middlewares/authMiddleware.ts`

Dependencias nuevas: `jsonwebtoken`, `@types/jsonwebtoken`

Documentación: `docs/TT-007-auth-jwt.md`

TT-008: Búsqueda y Filtros Avanzados

Objetivo: Ampliar `GET /api/propiedades` con nuevos filtros y ordenamiento dinámico.

Filtros agregados:

- `banios`, `parqueos`, `sector`, `ciudad`
- `metrajeMin`, `metrajeMax`
- `ordenarPor` (`precio`, `createdAt`, `metrajeTotal`)
- `ordenDireccion` (`asc`, `desc`)

Validación Zod: `filtrosPropiedadSchema.parse()` en el Service antes de llamar al repositorio.

Archivos modificados (3):

1. `application/validations/propiedad.validation.ts` — +8 campos al schema
2. `application/services/PropiedadService.ts` — validación Zod en `listar()`
3. `infrastructure/repositories/PropiedadRepository.ts` — filtros expandidos + `orderBy` dinámico

Ejemplo de uso:

```
GET /api/propiedades?
sector=Centro&banios=2&metrajeMin=100&ordenarPor=precio&ordenDireccion=asc
```

Documentación: `docs/TT-008-busqueda-filtros.md`

Resumen General

TT	Nombre	Archivos	Estado
001	Configuración del monorepo	6	✓
002	Esquema Prisma	2	✓
003	Conexión PostgreSQL	2	✓
004	CRUD Propiedades	11	✓
005	CRUD Asesores	10	✓
005.5	Refactor Hexagonal	12	✓

TT	Nombre	Archivos	Estado
006	Mensajes + Cloudinary	15	
007	Autenticación JWT	9	
008	Búsqueda y filtros	3	

Total archivos creados/modificados: ~70 archivos **Endpoints implementados:** 22 **Sprint:** 1 (completado)
Rol: Backend Lead / DevOps

Documento generado el 29/06/2026